

2IMV15 Simulation in Computer Graphics

Assignment 2 – Group 9

Lars van Gisbergen

`l.p.l.h.v.gisbergen@student.tue.nl`

Jeroen Oerlemans

`j.r.oerlemans@student.tue.nl`

Peter Heijstek

`p.j.heijstek@student.tue.nl`

I. INTRODUCTION

This report describes the mathematical foundation of the features implemented in the fluid simulation. The resulting code is not considered, as opposed to the report of the first assignment. Each feature is described in its own section. Those features are: vorticity confinement, fixed and moving objects, moving rigid bodies, two-way coupling, an attempt at combining the particle system of the previous assignment with the fluid simulation in this assignment, and lastly buoyancy. The personal contributions of the group members are listed in the appendix

II. VORTICITY CONFINEMENT

Fluid in motion usually displays vortices on different scales, however due to numerical dissipation, the small-scale vortices are damped and not at all noticeable in our simulation. This is especially the case in coarse grids. Vorticity confinement aims to mitigate this problem by introducing a corrective vorticity force for each cell. First, the “curl” of vector field $\mathbf{u}$ is found with $\omega = \nabla \times \mathbf{u}$. Then a gradient of the curl is calculated as $\eta = \nabla|\omega|$. The resulting vector points toward areas with high rotational strength. This vector is then normalized: $N = \frac{\eta}{|\eta|}$. The resulting confinement force is calculated as $f_{conf} = \epsilon h (N \times \omega)$. Essentially this is just the cross product of the normalized vector times the vorticity vector. The force causes the fluid to rotate around the N vector in each cell. The force is adjusted for the grid-size h . The amount of detail added back in the simulation is controlled by the ϵ parameter, which should be greater than zero.

The formulas above are continuous, but many simulations (ours included) use a discrete grid-based approach. To that end, the authors of Stam and Jensen [2] have provided discrete versions of the formulas above in the appendix. These formulas are taken and changed for 2 dimensions. The formula that calculates vorticity ω for some cell at i, j is

$$(\mathbf{u}_{i+1,j} - \mathbf{u}_{i-1,j} + \mathbf{v}_{i,j+1} - \mathbf{v}_{i,j-1})/2h.$$

The gradient is calculated with the central differences method. The formulas for this are as follows:

$$\eta_x = (\omega_{i-1,j} + \omega_{i+1,j})/2$$

$$\eta_y = (\omega_{i,j-1} + \omega_{i,j+1})/2$$

The vector η is then normalized into N , this is the same as in the continuous formulas. The cross product $N \times \omega$ is calculated per-coordinate as follows:

$$\begin{pmatrix} \eta_y \cdot \omega_{i,j} \\ -\eta_x \cdot \omega_{i,j} \end{pmatrix}$$

The final confining force is then found by simply multiplying this vector by the grid-size h and parameter ϵ .

III. FIXED OBJECTS

To let the fluid interact with its environment we first added immovable, solid obstacles. Each obstacle is an axis-aligned rectangle that marks cells the fluid cannot enter, so the flow must curve around it.

The key physical rule is the **no-slip boundary condition**: fluid in contact with a solid surface moves with that surface. Because our obstacles are stationary, the target velocity is zero. We enforce this directly on the MAC grid. At every time step, for each cell (i, j) inside a rectangle we set

$$u(i, j) = 0, \quad v(i, j) = 0.$$

We apply this overwrite just before the pressure–projection stage. Although the update itself is simple, realistic avoidance behaviour emerges during projection. Following Stam’s solver [1], [5], the method solves a Poisson equation for pressure, removing the artificial divergence introduced inside the obstacle. The resulting pressure gradient pushes the fluid sideways, yielding a plausible flow around the object.

IV. MOVING SOLID OBJECTS

We extend fixed obstacles to *movable* solids that the user can drag across the domain. The objects translate but do not rotate; their motion is dictated entirely by the mouse (one-way coupling).

The boundary still obeys the no-slip condition, yet now with a non-zero velocity equal to the object speed $\mathbf{v}^{\text{obj}}$. We compute

$\mathbf{v}^{\text{obj}}$ from the mouse displacement over the step Δt . For every grid cell (i, j) inside the object we enforce

$$u(i, j) = v_x^{\text{obj}}, \quad v(i, j) = v_y^{\text{obj}}.$$

These values are written just before the pressure–projection phase. The projection stage then removes the induced divergence, producing a pressure field that steers the fluid around the object. [1], [5].

V. MOVING RIGID BODIES

For more dynamic scenes we extend moving objects to full *rigid bodies*, adding rotation to translation. A rigid body never deforms, so the distance between any two of its points stays constant. Its motion is described by the centre-of-mass position $\mathbf{x}(t)$, orientation $\theta(t)$, linear velocity $\mathbf{v}(t)$ and angular velocity $\omega(t)$.

The user controls the body interactively: dragging sets its position, a key press toggles a constant spin, and releasing the mouse “throws” the body with a velocity taken from recent mouse motion. We advance the state each step with forward Euler:

$$\begin{aligned} \mathbf{x}(t + \Delta t) &= \mathbf{x}(t) + \mathbf{v}(t) \Delta t, \\ \theta(t + \Delta t) &= \theta(t) + \omega(t) \Delta t. \end{aligned}$$

Applying the no-slip condition to a shape that both translates and rotates is less trivial, because the footprint is neither static nor axis-aligned. For every grid-cell centre we first move it to body space—translate by $-\mathbf{x}(t)$, then rotate by $-\theta(t)$ —and use a simple axis-aligned box test there.

If a cell (i, j) lies inside the body we overwrite its velocity with the body’s linear velocity:

$$\begin{aligned} u(i, j) &= v_x(t), \\ v(i, j) &= v_y(t). \end{aligned}$$

Although we ignore the rotational term, the spinning outline still displaces fluid frame by frame, generating vortices. The pressure-projection step [1] removes the induced divergence and produces a convincing swirling wake. This yields a stable, visually plausible one-way coupling between the rigid body and the flow.

VI. COLLIDING CONTACT BETWEEN RIGID BODIES

To make the scene more dynamic, rigid bodies must interact not only with the fluid but also with each other. We implement collision handling for movable rectangles and disks. Resting contact is not required, so we use an impulse-based approach that separates overlapping bodies and updates their velocities in a plausible way. The procedure has two stages: *collision detection* and *collision response*.

A. Collision Detection

At every time step we test all pairs of movable objects. Three pair types are supported: rectangle–rectangle, disk–disk, and rectangle–disk.

Rectangle–Rectangle. For two convex rectangles we apply the *Separating Axis Theorem* (SAT). Two shapes are disjoint if there exists an axis on which their projections do not overlap. We check the axes perpendicular to the edges of both rectangles. If every axis overlaps, a collision has occurred. The axis with the smallest overlap gives the *minimum translation vector* (MTV) that resolves the penetration.

Disk–Disk. A collision occurs when the distance between centres $\mathbf{c}_A$ and $\mathbf{c}_B$ is smaller than the sum of radii:

$$\|\mathbf{c}_B - \mathbf{c}_A\| < r_A + r_B.$$

The MTV points from $\mathbf{c}_A$ to $\mathbf{c}_B$ with length

$$|\text{MTV}| = (r_A + r_B) - \|\mathbf{c}_B - \mathbf{c}_A\|.$$

Rectangle–Disk. We first find the point on the rectangle closest to the disk centre, testing the centre against the Voronoi regions of the rectangle’s edges and vertices. If the distance to this closest point is smaller than the disk radius, the bodies overlap. The MTV then points from the closest point to the centre, with magnitude equal to the penetration depth.

B. Collision Response

Given an MTV we resolve the contact in two steps and iterate the process to cope with multiple simultaneous collisions.

1. Positional correction. We separate the bodies along the MTV, sharing the shift according to inverse mass:

$$\Delta \mathbf{x}_A = \text{MTV} \frac{m_A^{-1}}{m_A^{-1} + m_B^{-1}}, \quad \Delta \mathbf{x}_B = -\text{MTV} \frac{m_B^{-1}}{m_A^{-1} + m_B^{-1}}.$$

A small tolerance is subtracted from the overlap to avoid immediate re-collision.

2. Impulse resolution. We then adjust the velocities. Let $\hat{\mathbf{n}}$ be the normal (the MTV direction) and

$$v_{\text{rel}} = (\mathbf{v}_B - \mathbf{v}_A) \cdot \hat{\mathbf{n}}$$

the relative normal speed. If $v_{\text{rel}} < 0$ the bodies are closing; the required impulse is

$$j = \frac{-(1 + \epsilon) v_{\text{rel}}}{m_A^{-1} + m_B^{-1}},$$

where $\epsilon \in [0, 1]$ is the coefficient of restitution. The velocity updates are

$$\Delta \mathbf{v}_A = -j m_A^{-1} \hat{\mathbf{n}}, \quad \Delta \mathbf{v}_B = j m_B^{-1} \hat{\mathbf{n}}.$$

This scheme conserves linear momentum and yields stable, believable collisions.

VII. TWO-WAY COUPLING

Two-way coupling is the implementation of objects not just influencing the fluid, but the fluid influencing the position of the object. For every object a few properties are defined

- p : grid position with p_x and p_y of the object
- w : width of the object
- h : height of the object
- v : velocity of the object
- m : mass of the object

for each of the obstacles, we want to calculate the average force between the edge points for both x and y sides. To do this we first calculate the edge points as such:

$$x_{min} = p_x - r, x_{max} = p_x + r$$

where $r = \frac{1}{2} * \sqrt{w^2 + h^2}$ for rectangles and simply r for disks. We calculate y_{min} and y_{max} the same way. By calculating the average flow between x_{min} and x_{max} and between y_{min} and y_{max} we derive u_f . Using this, we can calculate the force applied to the object as F_{drag} as:

$$F_{drag} = \gamma(u_f - v)$$

where γ is equal to the drag coefficient. Using this drag force, use Newtons second law to update the objects velocity:

$$\frac{dv}{dt} = \frac{1}{m} F_{drag} = \frac{1}{m} * \gamma(u_f - v)$$

Integrating time, we derive

$$v(t+1) = v(t) + \frac{1}{m} F_{drag} \Delta t = \frac{1}{m} \gamma(u_f - v(t)) \Delta t$$

This function updates the velocity of the object, thus two-way coupling an object based on the average fluid force applied to it.

VIII. PARTICLES AND FLUIDS

The particle system of the previous assignment was attempted in this new fluid model. However, due to some issues we encountered, it did not make the final version. We will however quickly discuss the implementation features and the potential fix to implement this fully.

By implementing the final model of the previous assignment, we can reintroduce the gravity- and springforces applied to the particles. Similarly the constraints that should work on the particles can be introduced. Using this, the particle system was reintegrated into the new fluid field. The particle system uses Explicit Euler as its integration scheme and uses the set Δt set for the fluid system.

For the interaction with the Cloth, the same functionality as two way coupling was introduces. Applying this formula which we defined in Section VI with every particle acting as an object:

$$v(t+1) = v(t) + \frac{1}{m} F_{drag} \Delta t = \frac{1}{m} \gamma(u_f - v(t)) \Delta t$$

to each of the particles, the individual particles should be updated by flow and use the spring forces applied to it to drag the cloth around the vector field.

The issue likely causing this not to function was that the particle system was using a different field than the fluid, making two distinct fields layered ontop of eachother with fluid grid $[u, v]$ and particle system on $[x, y]$. Thus the two-way coupling of the Cloth could not compute F_{drag} to update the particle nodes velocity.

IX. TEMPERATURE AND BUOYANCY

A. Temperature Field

Temperature T is stored on the same grid as density and velocity and follows

$$\partial_t T = -(\mathbf{u} \cdot \nabla) T + \kappa_T \nabla^2 T + S_T, \quad (1)$$

where κ_T is thermal diffusivity and S_T a user source (e.g. mouse-added heat). Implementation re-uses the scalar advection-diffusion kernels on a dedicated temperature array.

B. Buoyancy Force

An upward force

$$\mathbf{f}_{buoyancy} = \alpha (T - T_{ambient}) \hat{\mathbf{y}}, \quad (2)$$

with $T_{ambient}=0$ and user scale α , acts only on the vertical velocity:

$$v(i, j) \leftarrow v(i, j) + \Delta t \alpha T(i, j).$$

Applied before advection and projection, this produces rising plumes; vorticity confinement adds turbulence. Rendering maps T to colour, brightening hotter regions and leaving cooler ones grey.

REFERENCES

- [1] STAM, J. 1999. Stable Fluids. In *Computer Graphics (SIGGRAPH 1999)*, ACM, 121–128.
- [2] FEDKIW, R., STAM, J., AND JENSEN, H. 2001. Visual Simulation of Smoke. In *Computer Graphics (SIGGRAPH 2001)*, ACM, 15–22.
- [3] Rendering smoke and fire in real-time. http://graphics.ethz.ch/teaching/former/imagesynthesis_06/miniprojects/p3/index.html
- [4] ENRIGHT, D., MARSCHNER, S., AND FEDKIW, R. 2002. Animation and Rendering of Complex Water Surfaces. In *Computer Graphics (SIGGRAPH 2002)*, ACM, 736–744.
- [5] Jos Stam. Real-Time Fluid Dynamics for Games. *Proceedings of the Game Developer Conference*, March 2003.
- [6] Jos Stam, solver code, http://www.dgp.toronto.edu/people/stam/reality/Research/zip/CDROM_GDC03.zip.
- [7] Wikipedia, Gauss-Seidel method, https://en.wikipedia.org/wiki/Gauss%E2%80%93Seidel_method.
- [8] Robert Bridson, Fluid Simulation, https://www.cs.ubc.ca/~rbridson/fluidsimulation/2006/fluids_notes.pdf.

APPENDIX

All members have worked together in general, but to be more specific the personal contributions which each member worked the most on are listed below.

Lars has worked on:

- Fixed Objects
- Moving Solid Objects
- Moving Rigid Bodies
- Colliding Contact Between Rigid Bodies
- Temperature

Jeroen has worked on:

- Demo
- Two-Way Coupling
- Particle System Integration

Peter has worked on:

- Vorticity Confinement
- Colliding Contact Between Rigid Bodies
- Moving Rigid Bodies